

สคริปต์การสาธิตระบบและบทพูดอย่างละเอียด

โครงการ: ระบบร้านค้าออนไลน์ครบวงจร (E-Commerce Web Application)

⌚ เวลาสาธิต: 7 - 10 นาที • 🛠️ Stack: Laravel 11/13 + Bootstrap 5 + Supabase PostgreSQL + Docker • 🌐 Production: Render.com

✓ การเตรียมความพร้อมก่อนเริ่มสาธิต (Pre-Demo Setup Checklist)

1. เปิดเบราว์เซอร์ 2 หน้าต่างคู่กัน (Split Screen):

- หน้าต่างปกติ (Admin): ล็อกอิน admin@example.com / password เปิดค้างที่หน้า /admin
- หน้าต่าง Incognito (User): สำหรับลูกค้าทดสอบ user@example.com / password เปิดหน้าร้านค้า

2. URL ระบบ & Terminal:

- Cloud Production URL: <https://ecommerce-laravel-3usf.onrender.com>
- Local URL: <http://127.0.0.1:8000>
- Terminal: เปิดไฟล์เตอร์โปรเจกต์พร้อมรัน `php artisan test`

👥 1. ตารางสรุปการแบ่งงานและบทบาทหน้าที่ของสมาชิก (Work Breakdown)

ชื่อ / บทบาท	หน้าที่รับผิดชอบหลัก	ระบบและฟังก์ชันที่พัฒนาขึ้นจริง	ไฟล์โค้ดหลัก
1. หยาง Team Lead & DevOps	- System Architecture - Cloud Infrastructure - Auth & Security - Automated Testing	- ออกแบบสถาปัตยกรรมระบบ, ER Diagram และ Git Branching Workflow - เชื่อมต่อฐานข้อมูล Cloud PostgreSQL บน Supabase (Tokyo Pooler) - จัดทำ Dockerfile Multi-stage build, Apache config และ Reverse Proxy บน Render.com - แปลงระบบ Auth เป็น Bootstrap 5 (Login, Register, Reset) พร้อมบังคับ HTTPS ป้องกัน Mixed Content - เขียน Automated Test Suite ผ่าน 40/40 Tests (100%)	AppServiceProvider.php bootstrap/app.php layouts/guest.blade.php auth/login.blade.php Dockerfile, render.yaml

ชื่อ / บทบาท	หน้าที่รับผิดชอบหลัก	ระบบและฟังก์ชันที่พัฒนาขึ้นจริง	ไฟล์โค้ดหลัก
2. ฟิลิป Frontend Lead	- Main UI & Layout - Home Page - Shop Catalog - Search & Filtering	- ออกแบบธีมหลักของเว็บไซต์ (<code>layouts/app.blade.php</code>) ด้วย Bootstrap 5 และ Custom CSS - พัฒนาหน้าแรก (Home): Hero Banner, กล่อง 7 หมวดหมู่แบบ Card ดึงจำนวนสินค้าจริงแบบ Dynamic, สินค้าใหม่ 8 รายการ - พัฒนาหน้าร้านค้า (Shop Catalog): ระบบค้นหาซื้อสินค้า, ตัวกรองหมวดหมู่, ตัวกรองช่วงราคา (Min/Max), การจัดเรียงลำดับราคา/วันที่, และ Pagination แบบ Bootstrap 5 - พัฒนาหน้ารายละเอียดสินค้า (Product Detail): ภาพใหญ่, สต็อกจริง, ตัวเลือกจำนวนชิ้น, ปุ่มกดเพิ่มลดตะกร้า - พัฒนาหน้าติดต่อเรา (Contact Us): แสดงแผนที่และรายชื่อสมาชิกในทีม	<code>layouts/app.blade.php</code> <code>public/css/style.css</code> <code>HomeController.php</code> <code>ShopController.php</code> <code>home.blade.php</code> <code>shop/index.blade.php</code>
3. โชค Products Lead	- Database Design - Seeders & Mock Data - Admin Products CRUD - Thai Slug System	- ออกแบบตาราง <code>categories</code> และ <code>products</code> พร้อมความสัมพันธ์ Foreign Key - จัดทำ Seeders ข้อมูลจริง: <code>CategorySeeder</code> (7 หมวด) และ <code>ProductSeeder</code> (10 สินค้าจริง) - พัฒนาระบบจัดการสินค้าฝั่ง Admin (<code>ProductController</code>): ตารางสินค้า, ฟอर्मเพิ่มสินค้าพร้อมอัปโหลดรูปภาพเข้า <code>public/images/products</code>, แก้ไขสต็อกและราคา, ลบสินค้า - พัฒนาระบบจัดการหมวดหมู่ (<code>CategoryController</code>) - พัฒนา Thai Slug Fallback รองรับชื่อภาษาไทยใน URL	<code>migrations/*products.php</code> <code>migrations/*categories.php</code> <code>CategorySeeder.php</code> <code>ProductSeeder.php</code> <code>Admin/ProductController.php</code> <code>admin/products/*</code>
4. กวาง Cart & Orders Lead	- Cart Management - Checkout System - Stock Deduction Logic - DB Transaction	- ออกแบบตาราง <code>carts</code>, <code>orders</code>, <code>order_items</code> - พัฒนาระบบตะกร้าสินค้า (<code>CartController</code>): เพิ่มสินค้า, แสดงตัวเลข Badge บน Navbar แบบ Real-time, ปรับเพิ่ม-ลดจำนวน, ลบสินค้า, คำนวณยอดเงินรวม - พัฒนาระบบสั่งซื้อ (Checkout): ฟอर्मกรอกที่อยู่จัดส่งและเบอร์โทร, สรุปรายการสินค้า - พัฒนากระบวนการตัดสต็อกสินค้าด้วย Database Transaction (DB::transaction) ป้องกันสต็อกติดลบ - สร้างระบบ Generate รหัสคำสั่งซื้ออัตโนมัติ (<code>ORD-YYYYMMDD-XXXX</code>) และหน้าประวัติคำสั่งซื้อ	<code>migrations/*carts.php</code> <code>migrations/*orders.php</code> <code>Cart.php</code> , <code>Order.php</code> <code>CartController.php</code> <code>OrderController.php</code> <code>cart/index.blade.php</code> <code>cart/checkout.blade.php</code>

ชื่อ / บทบาท	หน้าที่รับผิดชอบหลัก	ระบบและฟังก์ชันที่พัฒนาขึ้นจริง	ไฟล์โค้ดหลัก
5. ปิงปอง User & Admin Ops	- User Profile - Admin Dashboard - Admin Order Management - User Management	- พัฒนาระบบโปรไฟล์ผู้ใช้งาน (ProfileController): แสดงข้อมูลส่วนตัว, ฟอรัมแก้ไขชื่อ/อีเมล, เปลี่ยนรหัสผ่านใหม่ - พัฒนาระบบ Admin Dashboard (DashboardController): แสดงการ์ดสถิติ Real-time 4 ด้าน (ยอดขายรวม, จำนวนออเดอร์, จำนวนสินค้า, จำนวนสมาชิก) - พัฒนาระบบจัดการคำสั่งซื้อสำหรับ Admin (Admin/OrderController): ดูรายการคำสั่งซื้อของลูกค้า, ดูรายละเอียดสินค้า, อัปเดตสถานะการจัดส่ง (รอชำระเงิน → จัดส่งแล้ว ฯลฯ) - พัฒนาระบบจัดการผู้ใช้ (Admin/UserController): ตรวจสอบรายชื่อและสิทธิ์การใช้งาน	ProfileController.php profile/show.blade.php Admin/DashboardController.php Admin/OrderController.php Admin/UserController.php admin/dashboard.blade.php admin/orders/*

2. สคริปต์การสาธิตอย่างละเอียดแบบแบ่งรายบุคคล (Minute-by-Minute Live Demo)

นาฬิกา 0:00 - 1:15

ผู้พูด: หยาง (Team Lead & DevOps)

ระบบที่พัฒนา: System Architecture, Supabase DB, Docker on Render

1. บทนำ, สถาปัตยกรรมระบบ และการแบ่งงานในทีม

🗨️ สิ่งที่ต้องทำบนหน้าจอ (Screen Actions):

- ยืนยันหน้าชั้นเรียน เปิดหน้าแรกของเว็บไซต์บน Render: <https://ecommerce-laravel-3usf.onrender.com>
- ชี้ให้อาจารย์เห็นแถบ URL ที่เป็น <https://> มีใบรับรองความปลอดภัย โลโก้ และเมนู Navbar สะอาดตา
- เลื่อนหน้าจอลงมาให้เห็นภาพรวมของระบบที่ตั้งข้อมูลจาก Cloud Database บน Supabase (Tokyo Region)

💬 บทพูด:

"กราบเรียนอาจารย์ที่เคารพและสวัสดีเพื่อนๆ ทุกคนครับ วันนี้กลุ่ม E-Commerce ของพวกเรา มีความยินดีที่จะนำเสนอและสาธิตการทำงานจริงของ 'ระบบร้านค้าออนไลน์ E-Commerce ครบวงจร' ครับ"

โปรเจกต์นี้ พวกเราทั้ง 5 คนได้ร่วมกันออกแบบและพัฒนาขึ้น โดยใช้ **Laravel** เป็น Backend Framework เชื่อมต่อกับฐานข้อมูล Cloud PostgreSQL บน **Supabase** และทำ Containerization ด้วย **Docker** นำขึ้นระบบ Cloud Production บน **Render.com** ซึ่งออนไลน์ใช้งานได้จริงแล้วในขณะนี้ครับ

ในการทำงาน พวกเราได้แบ่งความรับผิดชอบออกเป็น 5 ส่วนอย่างชัดเจน ได้แก่: 1. ผม (หยาง) ดูแลด้าน System Architecture, DevOps, Authentication และ Cloud Infrastructure 2. เพื่อนพลับ ดูแลด้าน Frontend UI, หน้าแรก และระบบค้นหาแคตตาล็อกสินค้า 3. เพื่อนโชค ดูแลโครงสร้างฐานข้อมูลสินค้า และระบบหลังบ้านจัดการสินค้าสำหรับ Admin 4. เพื่อนกวาง ดูแลระบบตะกร้าสินค้า ระบบ Checkout และ Logic การตัดสต็อกสินค้า 5. เพื่อนปิงปอง ดูแลระบบโปรไฟล์ผู้ใช้ และระบบจัดการออเดอร์สำหรับ Admin

ในลำดับแรกนี้ ขอส่งต่อให้ **เพื่อนพลับ** สาธิตการทำงานของฝั่งหน้าร้านและระบบค้นหาแคตตาล็อกสินค้าครับ"

2. ฟังก์ชัน: การเลือกชมสินค้า, ค้นหา และกรองช่วงราคา

🗨 สิ่งที่ต้องทำบนหน้าจอ (Screen Actions):

1. หน้าแรก (/): ให้เห็น Hero Banner, หมวดหมู่สินค้าทั้ง 7 หมวดหมู่พร้อมตัวเลขจำนวนสินค้าจริงจาก DB (เช่น เสื้อผ้า 1 ชิ้น, ของใช้ในบ้าน 2 ชิ้น) และการ์ดสินค้าใหม่ล่าสุด 8 รายการ
2. คลิกเมนู "สินค้า" (/shop): แสดงสินค้าทั้งหมดในรูปแบบ Grid Card สวยงาม พร้อมตัวแบ่งหน้า (Pagination)
3. ทดสอบระบบค้นหา (Search): พิมพ์คำว่า **เสื้อ** → หน้าจอรีเฟรชแสดงเฉพาะ "เสื้อยืดแขนสั้น Cotton 100%"
4. ทดสอบตัวกรองหมวดหมู่ (Category Filter): คลิกเลือกหมวดหมู่ **ของใช้ในบ้าน** → แสดง "ผงซักฟอก" และ "น้ำยาปรับผ้านุ่ม"
5. ทดสอบตัวกรองราคา (Price Filter): ป้อนราคา **50 ถึง 300** → ระบบแสดงเฉพาะสินค้าที่ตรงเงื่อนไขราคา
6. คลิกดูสินค้า "เสื้อยืดแขนสั้น Cotton 100%": เปิดหน้า /shop/t-shirt-cotton-100 ให้เห็นรูปภาพ รายละเอียด ราคา 350 บาท และป้ายสถานะสต็อกคงเหลือจริง (เช่น มีสินค้า 50 ชิ้น)

💬 บทพูด:

"สวัสดีครับอาจารย์ ผมฟิลิป รับผิดชอบในส่วนของ **Frontend UI และระบบแคตตาล็อกหน้าร้าน** ครับ"

สำหรับหน้าร้าน ผมได้ออกแบบ UI ด้วย **Bootstrap 5** โดยเน้นความสะอาดตา ใช้งานง่าย และเป็น Responsive รองรับทุกขนาดหน้าจอครับ • ในหน้าแรก เราจะเห็นแบนเนอร์โปรโมชัน และหมวดหมู่สินค้าทั้ง 7 หมวดหมู่ ซึ่งตัวเลขจำนวนสินค้าที่แสดงอยู่นี้ ถูกดึงและคำนวณสดจากฐานข้อมูลจริงผ่าน Eloquent Relationship ครับ

• เมื่อคลิกเข้ามาที่หน้า **'สินค้า' (Shop)** ผู้ใช้งานสามารถค้นหาสินค้าได้สะดวกรวดเร็ว เช่น เมื่อผมพิมพ์ค้นหาคำว่า 'เสื้อ' ระบบจะ Query กรองสินค้าที่ตรงกันมาแสดงทันที

• ด้านซ้ายมือจะมีตัวกรองหมวดหมู่ และตัวกรองช่วงราคา ซึ่งสามารถใช้งานร่วมกันได้ เช่น กรองหมวดของใช้ในบ้านที่มีราคาไม่เกิน 300 บาท

• เมื่อลูกค้าสนใจสินค้าชิ้นไหน สามารถคลิกเข้ามาที่ **'หน้ารายละเอียดสินค้า'** เพื่อดูข้อมูลเชิงลึก ราคา และสต็อกคงเหลือจริงในคลังได้ทันทีครับ

ลำดับต่อไป เมื่อลูกค้าถูกใจสินค้าแล้ว จะเข้าสู่กระบวนการสั่งซื้อ ขอส่งต่อให้ **เพื่อนกวาง** สาธิตระบบตะกร้าสินค้า และ Checkout ครับ"

3. ฟังก์ชัน: ระบบตะกร้าสินค้า, การ Checkout และการตัดสต็อกสินค้า

📋 สิ่งที่ต้องทำบนหน้าจอ (Screen Actions):

1. **หน้ารายละเอียดเสื้อยืด:** ปรับจำนวนเป็น 2 ชิ้น แล้วกดปุ่ม "เพิ่มลงในตะกร้า"
2. **ชี้ให้อาจารย์ดูที่ Navbar ด้านบน:** ตัวเลขสีแดง 2 ปรากฏขึ้นบนไอคอนตะกร้าทันที
3. **คลิกเปิดหน้า "ตะกร้าสินค้า" (/cart):** แสดงเสื้อยืด 2 ตัว x 350 บาท = 700 บาท ลองกดเพิ่มเป็น 3 ชิ้น แล้วกดอัปเดต → ยอดรวมเปลี่ยนเป็น 1,050 บาท
4. **คลิกปุ่ม "ไปหน้าสั่งซื้อสินค้า" (/checkout):** กรอกข้อมูลจัดส่ง: ชื่อ "สมชาย ใจดี", เบอร์โทร "081-234-5678", ที่อยู่ "เชียงใหม่" → กดปุ่ม "ยืนยันการสั่งซื้อ"
5. **แสดงหน้าผลลัพธ์การสั่งซื้อ (/orders/{id}):** ชี้ให้เห็นรหัสคำสั่งซื้ออัตโนมัติ (เช่น ORD-20260923-XXXX) สถานะเริ่มต้นเป็นสีเหลือง "รอชำระเงิน" และสต็อกสินค้าถูกตัดออกไป 3 ชิ้นเรียบร้อยแล้ว

💬 unพูด:

"สวัสดีครับอาจารย์ ผมกวาง รับผิดชอบในส่วนของ ระบบตะกร้าสินค้า (Cart) และขั้นตอนการสั่งซื้อ (Checkout) ครับ"

จุดเด่นในส่วนที่ผมพัฒนา มีดังนี้ครับ: 1. เมื่อลูกค้ากดเพิ่มสินค้าลงตะกร้า ระบบจะส่งคำขอไปยัง **CartController** เพื่อบันทึกข้อมูล และมี Badge สีแดงแสดงจำนวนสินค้าบน Navbar อัปเดตให้เห็นทันที เพื่อให้ลูกค้ารู้สถานะตะกร้าของตัวเองตลอดเวลา

2. ในหน้าตะกร้า ลูกค้าสามารถเพิ่ม-ลดจำนวนชิ้น หรือลบสินค้าที่ไม่ต้องการออกได้ โดยระบบจะคำนวณยอดเงินรวมใหม่อัตโนมัติ

3. ในขั้นตอน **Checkout (สั่งซื้อสินค้า)** จุดนี้เป็นหัวใจสำคัญด้านความถูกต้องของข้อมูล ผมได้เขียนการทำงานไว้ภายใต้ **Database Transaction (DB::transaction)** ครับ หมายความว่าเมื่อลูกค้ากดยืนยันสั่งซื้อ ระบบจะทำงาน 3 อย่างพร้อมกันในคราวเดียว: - บันทึกข้อมูลใบสั่งซื้อลงตาราง **orders** พร้อม Generate เลขที่ใบสั่งซื้อเฉพาะ - นำสินค้าในตะกร้ามาบันทึกลงตาราง **order_items** พร้อมล็อกราคา ณ วันที่ซื้อ - ทำการ **ตัดสต็อกสินค้า** ในตาราง **products** ทันที และล้างตะกร้าสินค้าออก - หากมีขั้นตอนใดเกิดปัญหา ระบบจะ Rollback คืนค่าทั้งหมดทันที ทำให้สต็อกไม่ผิดพลาดและไม่เกิดปัญหาสต็อกติดลบครับ

หลังจากสั่งซื้อเรียบร้อยแล้ว สถานะเริ่มต้นของออเดอร์จะเป็น 'รอชำระเงิน' ต่อไปจะส่งต่อให้ **เพื่อนโชค และเพื่อนปิงปอง** รับผิดชอบการจัดการในฝั่งหลังบ้าน Admin ครับ"

4. ฝั่งหลังบ้าน Admin: การจัดการสินค้า, ตรวจสอบสต็อก และอัปเดตสถานะคำสั่งซื้อ

📋 สิ่งที่ต้องทำบนหน้าจอ (Screen Actions):

1. สลับไปที่หน้าต่างบราวเซอร์ที่ 1 (ล็อกอินเป็น Admin ค้างไว้):
2. ปิงปองเปิดหน้า Admin Dashboard (`/admin`): ชี้ให้เห็นการ์ดสถิติ 4 ใบ ยอดขายรวมเพิ่มขึ้นตามยอด 1,050 บาทที่กว้างเพ่งสั่งซื้อ, ออเดอร์เพิ่มขึ้น 1 รายการ
3. โฆษเปิดหน้าจัดการสินค้า (`/admin/products`): ชี้ให้ดูแถวของ "เสื้อยืดแขนสั้น Cotton 100%" สต็อกลดลงจาก 50 เหลือ 47 ชิ้น ตรงตามที่กว้างกดสั่งซื้อไป 3 ชิ้นจริง และเปิดหน้าฟอร์มแก้ไข/เพิ่มสินค้าพร้อมระบบอัปโหลดรูปภาพ
4. ปิงปองเปิดหน้าจัดการคำสั่งซื้อ (`/admin/orders`): พบออเดอร์ล่าสุดของ "สมชาย ใจดี" อยู่บนสุด → คลิกดูรายละเอียด → ในกล่องอัปเดตสถานะ เปลี่ยนจาก "รอชำระเงิน" เป็น "จัดส่งแล้ว" (Shipped) แล้วกดบันทึก → ป้าย Badge เปลี่ยนเป็นสีน้ำเงินทันที

💬 unพูด:

โฆษ: "สวัสดีครับอาจารย์ ผมโฆษ รับผิดชอบในส่วนของ **ฐานข้อมูลสินค้า และระบบจัดการสินค้าหลังบ้าน (Products Management)** ครับ"

- ในฝั่ง Admin เราสามารถตรวจสอบรายการสินค้าทั้งหมดในคลังได้ โดยสต็อกจะซิงค์กับฝั่งลูกค้าแบบ Real-time อย่างที่เห็นครับ เมื่อสักครู่งานสั่งซื้อเสื้อยืดไป 3 ตัว สต็อกในหน้าหลังบ้านนี้ก็ปรับลดลงเหลือ 47 ตัวทันทีโดยไม่ต้องรีเซ็ตรบบ
- Admin สามารถเพิ่มสินค้าใหม่ อัปเดตจำนวนสต็อก ปรับเปลี่ยนราคา และอัปโหลดรูปภาพเข้าสู่เซิร์ฟเวอร์ได้อย่างสะดวกรวดเร็ว
- นอกจากนี้ ผมได้พัฒนาฟังก์ชัน **Thai Slug Fallback** เพื่อแก้ปัญหาภาษาไทยใน URL ทำให้สินค้าที่ตั้งชื่อเป็นภาษาไทยสามารถแสดงผล URL ได้อย่างถูกต้อง สวยงาม และเป็นมิตรกับ SEO ครับ"

💬 unพูด:

ปิงปอง: "สวัสดีครับอาจารย์ ผมปิงปอง รับผิดชอบในส่วนของ **Admin Dashboard และระบบจัดการคำสั่งซื้อ (Orders Management)** ครับ"

- หน้า Dashboard หลังบ้านนี้ถูกพัฒนาขึ้นเพื่อให้ผู้บริหารร้านค้าเห็นภาพรวมธุรกิจได้ทันที ทั้งยอดขายสะสม จำนวนออเดอร์ และจำนวนสมาชิก
- และในหน้า 'จัดการคำสั่งซื้อ' แอดมินสามารถเข้ามาตรวจสอบใบสั่งซื้อของลูกค้าได้ละเอียดแบบรายบิล พร้อมทั้งอัปเดตสถานะการส่งสินค้า เช่น ตอนนี้ผมได้เปลี่ยนสถานะออเดอร์ของคุณสมชายเป็น 'จัดส่งแล้ว' เพื่อให้ลูกค้าสามารถเช็คสถานะจากฝั่งหน้าบ้านได้ทันทีครับ"

5. ตรวจสอบผลลัพธ์รอบด้าน, การทดสอบอัตโนมัติ และสรุปภาพรวม

🗨️ สิ่งที่ต้องทำบนหน้าจอ (Screen Actions):

1. สลับกลับมาที่หน้าต่างบราวเซอร์ที่ 2 (ฝั่งลูกค้าคุณสมชาย): รีเฟรชหน้า "คำสั่งซื้อ" (/orders) → แสดงสถานะเปลี่ยนเป็นสีน้ำเงิน "จัดส่งแล้ว" ตามที่แอดมินอัปเดต
2. เปิดหน้าโปรไฟล์และ Auth (/profile, /login): แสดงดีไซน์ Bootstrap 5 สวยงาม การ์ดมุมมน rounded-4 และปลอดภัยด้วย HTTPS
3. สลับไปหน้าต่าง Terminal: รันคำสั่ง `php artisan test` → หน้าจอแสดงแถบสีเขียว **PASS: 40 passed (110 assertions)** ครบ 100%
4. เปิดหน้า "ติดต่อเรา" (/contact): แสดงรายชื่อและบทบาทสมาชิกในทีมทั้ง 5 คน

💬 บทพูด:

"กลับมาที่ฝั่งลูกค้าครับ เมื่อเรากลับมารีเฟรชหน้า 'ประวัติคำสั่งซื้อ' จะเห็นว่าสถานะคำสั่งซื้อเปลี่ยนเป็น 'จัดส่งแล้ว' อย่างถูกต้อง สอดคล้องกับการทำงานแบบ End-to-End ตั้งแต่หน้าร้านจนถึงหลังบ้านครับ"

และเพื่อเป็นการยืนยันความสมบูรณ์และเสถียรภาพของโค้ดทั้งหมดที่พวกเราทั้ง 5 คนร่วมกันเขียนขึ้น ผมได้เขียน **Automated Test Suite (PHPUnit Tests)** ครอบคลุมทุกระบบหลัก ทั้งระบบความปลอดภัย ระบบสินค้า ระบบตะกร้า และระบบคำสั่งซื้อ

ดังที่อาจารย์เห็นบนหน้าจอ Terminal ครับ ผลการรัน `php artisan test` ผ่านครบถ้วนทั้ง **40 Tests 110 Assertions คิดเป็น 100% เต็ม** ไม่มีข้อผิดพลาดใดๆ ทั้งสิ้นครับ

และระบบนี้ได้ถูก Deploy ขึ้นสู่ Cloud Production บน **Render.com** เชื่อมต่อกับ Cloud Database บน **Supabase** ทำให้ทุกคนสามารถเข้าใช้งานระบบร้านค้าออนไลน์ผ่านอินเทอร์เน็ตได้จริงทุกที่ทุกเวลาครับ

กลุ่ม E-Commerce ของพวกเรา ขอจบการสาธิตการทำงานของระบบไว้แต่เพียงเท่านี้ และพวกเราทุกคนพร้อมตอบข้อซักถามของอาจารย์แล้วครับ ขอขอบพระคุณครับ"

? 3. แนวทางการตอบคำถามอาจารย์แบบแยกรายบุคคล (Q&A Preparation)

หยาง (DevOps & Auth)

คำถาม: จัดการปัญหาด้านความปลอดภัยและระบบ Auth อย่างไรเมื่อขึ้น Cloud?

คำตอบ: "เราใช้ Laravel Breeze เป็นฐาน แต่ได้ปรับแต่งให้เป็น Bootstrap 5 ทั้งหมด พร้อมตั้งค่า `trustProxies(at: '*')` และ `URL::forceScheme('https')` เพื่อให้ทำงานร่วมกับ SSL Reverse Proxy บน Render ได้อย่างปลอดภัย ป้องกันปัญหา Mixed Content และมีการเข้ารหัสที่ปลอดภัยด้วย Bcrypt / Argon2ID รวมทั้งมี CSRF Protection ป้องกันการโจมตีทุกฟอร์มครับ"

พลับ (Frontend)

คำถาม: หน้าร้านรองรับ Responsive อย่างไร และดึงข้อมูลหมวดหมู่มาแสดงอย่างไร?

คำตอบ: "เราใช้ Grid System ของ Bootstrap 5 (`col-lg-*`, `col-md-*`) ร่วมกับ Custom CSS ทำให้หน้าเว็บปรับขนาดตามอุปกรณ์ทั้งมือถือและคอมพิวเตอร์อย่างสวยงาม ส่วนหมวดหมู่สินค้า เราใช้ Eloquent Query `Category::withCount('products')->get()` ทำให้ดึงหมวดหมู่พร้อมจำนวนสินค้าจริงมาแสดงได้ใน Query เดียว ช่วยลดปัญหา N+1 Query ครับ"

โชค (Products)

คำถาม: แก้ปัญหาเรื่องรูปภาพสินค้าและการตั้งชื่อภาษาไทยอย่างไร?

คำตอบ: "สำหรับการอัปโหลดรูปภาพ เราเขียนโค้ดตรวจสอบและสร้างโฟลเดอร์ `public/images/products` อัตโนมัติ พร้อมตั้งชื่อไฟล์ใหม่ด้วย `time().'_'.uniqid()` ป้องกันชื่อไฟล์ซ้ำ ส่วนชื่อภาษาไทย ฟังก์ชัน `Str::slug()` ของ Laravel จะคืนค่าว่าง เราจึงเขียน Fallback Logic ให้สร้าง slug แบบ `product-` ตามด้วยเวลา และตรวจสอบความซ้ำใน DB เพื่อให้ได้ URL ที่ไม่ชนกันครับ"

กวาง (Cart & Orders)

คำถาม: ถ้ามีลูกค้ากดสั่งซื้อสินค้าชิ้นสุดท้ายพร้อมกัน 2 คน จะเกิดอะไรขึ้น?

คำตอบ: "ระบบของเราใช้ Database Transaction (`DB::transaction()`) ในการสั่งซื้อครับ เมื่อลูกค้ากดสั่งซื้อ ระบบจะตรวจเช็คสต็อก ณ วินาทีนั้น หากสต็อกไม่เพียงพอ ระบบจะโยน Exception และแจ้งเตือนลูกค้าว่าสินค้าไม่พอ พร้อม Rollback ยกเลิกรายการสั่งซื้อทั้งหมดทันที ทำให้สต็อกไม่มีความผิดพลาด และไม่เกิดข้อผิดพลาดในการตัดเงินหรือสร้างใบสั่งซื้อครับ"

ปิงปอง (User & Admin)

คำถาม: ป้องกันไม่ให้ลูกค้าทั่วไปแอบเข้าหน้า Admin ได้อย่างไร?

คำตอบ: "เราสร้าง `AdminMiddleware` ขึ้นมาคุ้มครอง Route กลุ่ม `/admin` ทั้งหมดครับ โดยตัว Middleware จะตรวจสอบ 2 ขั้นตอน 1. ต้องผ่านการล็อกอิน (`auth`) และ 2. เมธอด `$user->isAdmin()` ต้องคืนค่าเป็น `true` (เช็คฟิลด์ `is_admin == 1`) หากลูกค้าทั่วไปพยายามเข้า URL หลังบ้าน ระบบจะปฏิเสธและเตือน 'Unauthorized Access' ทันทีครับ"